Revision :-

N bit $\rightarrow (N-1)$

Q1) 8 bit :

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	1	0	0	0	1	1	0

:- $-2^7 + 2^6 + 2^2 + 2^1$

$\Rightarrow \underline{-58}$

Q2) 4 bit :

2^3	2^2	2^1	2^0
1	0	1	0

$= -2^3 + 2^1 = -6$

Q3) 8 bit :

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	0	0	0	1	0	1	0

$= 2^3 + 2^1 = \underline{10}$

Q4) N bits :-

0	1	1	1	1	1	-	-	-	-	1
	1	0	0	0	0	0	.....			0
	(N-1)									
	⊖ive							⊕ive	3	2	1	0

Min value – -2^{N-1}

Max Value – $\underline{2^0 + 2^1 + 2^2 + \cdots 2^{N-2}} \rightarrow 2^{N-1} - 1$

Range – $[-2^{N-1}, 2^{N-1} - 1]$

$a \wedge a = 0$

$a \wedge b$:→ if both bits are different ⇒ 1

same ⇒ 0

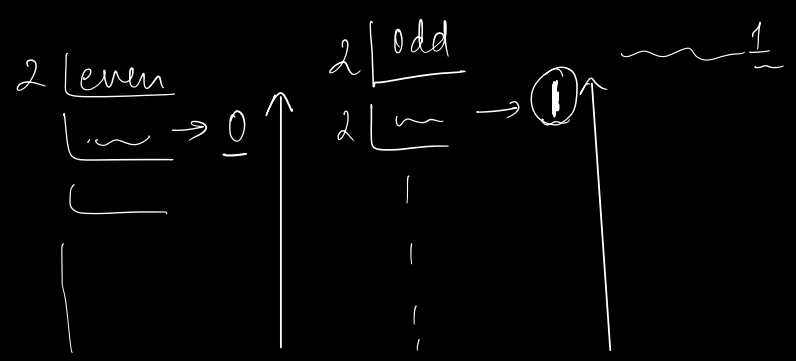
Bitwise Operators :- { &, |, ^, ~, <<, >> }

byte a = 28 :	0	0	0	1	1	1	0	0	
byte b = 18 :	0	0	0	1	0	0	1	0	
a & b :	0	0	0	1	0	0	0	0	(16)
a | b :	0	0	0	1	1	1	1	0	(30)
a ^ b :	0	0	0	0	1	1	1	0	
~a :	1	1	1	0	0	0	1	1	

& (if both are set - 1 else - 0 }

| (if any of them is set)

^ (both bits should be diff)



int a = 10 } $a \& 1 = 0$

$a = 0\ 0\ 0\ 0\ 1\ 0\ 1\ 0$

$1 = 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1$

$a \& 1 = \underbrace{0\ 0\ 0\ 0\ 0\ 0\ 0\ 0}_{0}$

int a = 11 } $a \& 1 = 1$

$a = 1\ 0\ 1\ 1$

$1 = 0\ 0\ 0\ 1$

$a \& 1 = \underbrace{0\ 0\ 0\ 1}_{1}$

0th bit is always set in Odd Nos
unset in even Nos. }

0th bit $\Rightarrow 0$ } $a \& 1 = 0$

0th bit $\Rightarrow 1$ } $a \& 1 = 1$

```
if (a & 1 == 0) {
    0th bit is unset
}
```

even

```
if (a & 1 == 1) {
    0th bit is set
}
```

odd

Properties :-

$a \& 0$:-

$a \& a$:-

$a \mid 0$:-

$a \mid a$:-

$a \wedge 0$:-

$a \wedge a$:-

$a \wedge a = 0$

$a \wedge 0 = a$

$$\begin{array}{r} a = 1010 \\ 0 = 0000 \\ \hline 1010 \end{array}$$

Q1) $a \wedge b \wedge b \wedge c \wedge a$:- $\underbrace{a \wedge a}_{0} \wedge \underbrace{b \wedge b}_{0} \wedge c \Rightarrow 0 \wedge c$

$\Rightarrow c$.

10 5 5 6 10 $\Rightarrow \cancel{10 \wedge 10}^{0} \wedge \cancel{5 \wedge 5}^{0} \wedge 6^{6} \Rightarrow 6$.

Q2) $c \wedge d \wedge a \wedge c \wedge a \wedge g \wedge d$:- $\underbrace{a \wedge a}_{0} \wedge \underbrace{c \wedge c}_{0} \wedge \underbrace{d \wedge d}_{0} \wedge g$

g

Q1) Given N array elements, every element repeats 2 times except 1 element, find unique element?

Ex ar[7]: 3 8 4 8 3 7 4 → ans = 7

Ex ar[5]: 2 9 7 2 7 → ans = 9

xor = 0

2 11 12 14 9 {xor is commutative}

Idea:- get xor of all elements

```
ans = 0
for(int i=0; i<N; i++){
    ans = ans ^ arr[i]
}
return ans
```

Left Shift (<<) $\} \rightarrow \{-128, 127\}$

byte a = 10

	-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
	0	0	0	0	1	0	1	0	$\rightarrow \underline{10}$
a << 1 :- (discard)	0	0	0	1	0	1	0	0	$\} \; 16+4=20 \rightarrow 10\times 2^1$
a << 2 :- (discard)	0	0	1	0	1	0	0	0	$\} \; 32+8=40=10\times 2^2$
a << 3 :- (discard)	0	1	0	1	0	0	0	0	$\} \; 64+16=80=10\times 2^3$
a << 4 :- (discard)	1	0	1	0	0	0	0	0	$\}$ Expected val : $10\times 2^4=160$

$\hookrightarrow -2^7+2^5 = -128+32$

$\Rightarrow \boxed{-96}$

Ideally this should have been 160, but as 160 cannot be stored in 8 bits. Hence overflow

$a << 1 = a*2 = a\times 2^1$

$a << 2 = a*2*2 = a\times 2^2$

$a << 3 = a*2*2*2 = a\times 2^3$

$a << N = a*2^N$.

Imagine that there is no overflow,

$$\boxed{a << N = a * 2^N}$$

$a << 1 = a * 2^1$

$a << 2 = a * 2^2$

$a << 3 = a * 2^3$

$\underline{2^N}$

$a << N = a * 2^N$

Put $\underline{a=1}$

$1 << N = 1 * 2^N$

$$\boxed{2^N = 1 << N}$$

Input :-

Find 2^N

return $(1 << N)$

int(32)

$[-2^{31}, 2^{31}-1]$

$N = 29 = 2^{29}$ } ✓

$N = 30 = 2^{30}$ } ✓

$N = 31 = 2^{31}$ } Overflow

64 → $[-2^{63}, 2^{63}-1]$

return $(1LL << N)$

long data type

$N = 31 \rightarrow 2^{31}$ ✓

$N = 60 \rightarrow 2^{60}$ ✓

$N = 63 \rightarrow 2^{63}$ ✗

$> 2^{62}$

$N > 62$:-

1) Python :- { No issue } → string

2) Java : { Big Integer }
↳ Library | functions

$5^N = 5 << N$

$1 << N = 2^N$

$a << N = a * 2^N$ Put $a = 5$ } → Mistakes *

$5 << N = 5 * 2^N$

Right Shift :- (>>)

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
byte a = 50 :	0	0	1	1	0	0	1	0	discard
a >> 1 :	0	0	0	1	1	0	0	1	$16+8+1=25=\frac{50}{2}$
a >> 2 :	0	0	0	0	1	1	0	0	$8+4=12=\frac{50}{4}$
a >> 3 :	0	0	0	0	0	1	1	0	$4+2=6=\frac{50}{8}$
a >> 4 :							1	1	$2+1=3=\frac{50}{16}$
a >> 5 :								1	$1=\frac{50}{32}$
a >> 6 :									$=0=\frac{50}{64}$
a >> 7									0
a >> 8									0

$$a >> N = \frac{a}{2^N}$$

$$a << N = a * 2^N$$
$$a >> N = a/2^N$$

>> <<

Problem :-

Given N & i, check if i^{th} bit position in number N is set or not? $\{0 <= N <= 10^9\}$ $\{0 <= i <= 30\}$

	4	3	2	1	0
N = 26 →	1	1	0	1	0

i = 2 → return false

	5	4	3	2	1	0
N = 35 →	1	0	0	0	1	1

i = 1 → return true

150

	7	6	5	4	3	2	1	0
N :-	1	0	0	1	0	1	1	0
N>>1 :	0	1	0	0	1	0	1	1
N>>2 :	0	1	0	0	1	0	1	

i = 0 ? $\boxed{N \& 1 == 1}$ } 0

i = 1 ? $\boxed{(N >> 1) \& 1 == 1}$ } 1

i = 2 ? $(N >> 2) \& 1 == 1$ }

i = 3 ? $(N >> 3) \& 1 == 1$ }

3rd bit in N will come at 0th bit

```
bool checkBit(int N, int i){
    if ((N >> i) & 1 == 1){
        return true
    }
    else{
        return false
    }
}
```

Given N, set i^{th} bit in N :-

Note :- If i^{th} bit in N is already set, no changes in N.

$N = 26$:

2^4	2^3	2^2	2^1	2^0
4	3	2	1	0
1	1	0	1	0

$i = 2$

$\downarrow$

Set 2nd bit in N : 1 1 1 1 0 } 30

2^5	2^4	2^3	2^2	2^1	2^0
5	4	3	2	1	0
1	0	0	0	1	1
1	0	1	0	1	1

$N = 35 \rightarrow$

$i = 3 \rightarrow$

$35 + 2^3 \Rightarrow \underline{43}$

i^{th} bit $\rightarrow$ $\underline{N + 2^i}$

	3	2	1	0
$N = 4 \rightarrow$	0	1	0	0
$i = 2 \rightarrow$	0	1	0	0

$\rightarrow$ ans = 4

	7	6	5	4	3	2	1	0
N =	1	0	0	1	0	1	1	0
$i = 4$:	0	0	0	1	0	0	0	0
$N \mid (1 << i)$:	1	0	0	1	0	1	1	0

	7	6	5	4	3	2	1	0
N =	1	0	0	1	0	1	1	0
$i = 5$:	0	0	1	0	0	0	0	0
	1	0	1	1	0	1	1	0

```
int Set (int N, int i) {
    if(checkBit(N, i)) {
        return N
    }
    else {
        return N + (1<<i)
    }
}

return N | (1<<i);
```

already written this

N: i^{th} bit is set → Set

N: i^{th} is unset → Set

N → set i^{th} bit

2^i

$(N \mid 2^i) \rightarrow 1$

	2^4	2^3	2^2	2^1	2^0
	4	3	2	1	0
N = 26 :	1	1	0	1	0
i = 3	0	1	0	0	0
ans →	1	1	0	1	0

1 1 0 → 1

0 1 0 → 0

Q) Set x^{th} & y^{th} bit in number 0.

|| Constraints :- x^{th} & y^{th} < 30

Input : $x = 2,\ y = 5$ →

$$\begin{array}{r} 100000 \\ 000100 \\ \hline 100100 \end{array}$$

5 4 3 2 1 0

1 0 0 1 0 0 → $2^5 + 2^2 = \underline{36}$

Input :- $x = 3,\ y = 4$ →

$$\begin{array}{r} 01000 \\ 10000 \\ \hline 11000 \end{array}$$

4 3 2 1 0

1 1 0 0 0 → $2^3 + 2^4 = \underline{24}$

```
setBit(int x, int y) {
    return (1 << x) + (1 << y);
}
```

(1<<x) | (1<<y)

$x = 2,\ y = 2$

1<<2 : 1 0 0

1<<2 : 1 0 0

$$\begin{array}{r} 100 \\ \hline 100 \end{array} \rightarrow 2^2 \Rightarrow \underline{4}$$

$x = 2,\ y = 2$

↓

8

$$\begin{array}{r} 10 \\ 100 \\ \hline \end{array}$$

↳ $\underline{4}$